Tamil Political Social Media Content Emotion Recognition

Jyothsna Vaasudevan, Harish Manukonda

***Abstract*—The realm of politics has always been a hotbed of diverse opinions and sentiments. In the contemporary era, characterized by increasing social media usage and widespread citizen engagement, political discourse has expanded beyond traditional channels and permeates social media, news platforms, and online forums. Tamil-speaking regions, known for their dynamic political landscape, produce substantial amounts of textual data daily. Analyzing this data can reveal valuable insights into public sentiment regarding policies, leaders, and sociopolitical trends. However, Tamil, being a low-resource language, presents unique challenges such as linguistic nuances, dialectical variations, and a scarcity of annotated data. This study endeavors to bridge these gaps by building an advanced machine learning framework for classifying Tamil political text into multiple sentiment categories. This initiative provides tools and insights for policymakers, researchers, and media outlets, enabling a deeper understanding of public sentiment in Tamil-speaking regions. Keywords - Emotion Recognition, mBERT, Political Social Media Content Tamil Text, SVM, XGBoost, Random Forest.**

I. Introduction

In the field of natural language processing (NLP), subjective text analysis has become a key approach for understanding human attitudes, opinions, and emotions expressed in written language. Two important tasks within this domain are sentiment analysis and emotion recognition. While both sentiment analysis and emotion recognition aim to extract subjective information from text, they differ significantly in their scope, complexity, and application. Sentiment analysis primarily focuses on determining the overall attitude or polarity of the text, typically categorizing it into broad groups such as positive, negative, and neutral. This approach is widely used in areas like customer feedback evaluation, social media monitoring, and brand sentiment analysis, where businesses need a general sense of public perception toward a product, service, or topic.

Emotion recognition, on the other hand, seeks to identify the specific emotions expressed in a piece of text, such as happiness, sadness, anger, fear, surprise, or disgust. Unlike sentiment analysis, which deals with broad categories, emotion recognition is more fine-grained, as it must differentiate between multiple nuanced emotional states. For instance, while both anger and disappointment may contribute to negative sentiment, they represent distinct emotional responses that require deeper analysis such as its applications in mental health monitoring, personalized user experiences, and human-computer interaction, where precise emotional interpretation is crucial.

The distinction between these two tasks becomes especially important in the context of political discourse. This study focusses on Tamil political emotion recognition which requires emotion recognition instead of sentiment analysis because political discourse often conveys a wide range of emotions beyond simple positive, negative, or neutral sentiments. In political contexts, people do not merely express whether they feel positively or negatively about a topic; rather, they convey specific emotions such as anger, fear, hope, trust, disappointment, or enthusiasm. For instance, a statement about a government policy may express anger due to dissatisfaction, fear due to uncertainty, or hope for a positive change. Sentiment analysis would simply label such statements as ”negative” or ”positive,” failing to distinguish between these crucial emotional states. Recognizing these fine-grained emotions is essential for understanding public perception, political polarization, and voter sentiment in a more precise manner.

By employing emotion recognition instead of sentiment analysis, this research aims to provide a more detailed and accurate understanding of emotional responses to political events, leaders, and policies. This deeper insight can assist political analysts, policymakers, and researchers in identifying patterns of public sentiment, tracking political polarization, and detecting shifts in voter attitudes. Ultimately, the ability to recognize specific emotions in political discourse offers a powerful tool for understanding the evolving dynamics of political communication and public engagement.

II. Literature review

Several studies have explored the challenges and advancements in sentiment analysis for non-English languages, emphasizing the role of deep learning and preprocessing techniques. The review on non-English sentiment analysis [1] highlights the scarcity of linguistic resources, datasets, and models, which limits performance compared to English. However, advances in multilingual embeddings and transfer learning have improved accuracy across various languages. One approach leverages multilingual transformer models like XLM-RoBERTa combined with automatic translation for data augmentation, demonstrating improved performance on sentiment analysis tasks for non-English tweets [2]. This method effectively addresses the lack of annotated corpora in low-resource languages by augmenting training data through machine translation.

Studies on Arabic sentiment analysis [3] address dialectal variations and complex morphology, demonstrating the effectiveness of deep learning techniques in handling linguistic diversity. Similarly, document-level sentiment analysis in Telugu [4] underscores the superiority of deep learning over traditional methods, emphasizing the importance of context-aware models

and effective feature extraction for better domain adaptability. Research on Tamil code-mixed text further extends sentiment analysis by tackling spelling variations and syntactic inconsistencies, a common issue in social media and informal text. The use of pre-trained ULMFiT models [5] enhances classification accuracy, showcasing the benefits of transfer learning in low-resource languages. Recent work has also highlighted the efficacy of transformer-based architectures like BERT in capturing nuanced emotions and sentiments from text data. Fine-tuning BERT models for both sentiment analysis and emotion recognition tasks has yielded high accuracy on real-world tweet datasets, especially when handling noisy, unstructured text such as tweets containing slang, abbreviations, and typos [6].

One study proposed a novel sentiment analysis model combining a pre-trained BERT-large-cased (BLC) model with a Decision-Based Recurrent Neural Network (D-RNN). This approach integrates advanced preprocessing techniques (e.g., tokenization, stopword removal) alongside feature extraction methods like Bag-of-Words (BoW) and Word2Vec embeddings to enhance classification accuracy. The model's aspect-based and priority-based sentiment classification strategies address challenges such as noisy datasets and domain-specific sentiment patterns effectively [7].

Another study introduced a feature ensemble model to improve tweet sentiment analysis, particularly for tweets containing fuzzy sentiments (e.g., ambiguous or context-dependent phrases). This method combines lexical, semantic, positional, and polarity-based features into tweet embeddings processed using a CNN model. The approach significantly enhances performance metrics such as precision and F1-score compared to traditional embedding methods like GloVe or Word2Vec [8]. By focusing on fuzzy sentiments through specialized tweet embeddings and leveraging convolutional neural networks (CNNs), this study demonstrates how tailored feature extraction can address challenges unique to informal social media text.

Across these studies, deep learning significantly outperforms traditional approaches when combined with rigorous preprocessing techniques such as tokenization, stemming, and stopword removal. Despite advancements, challenges remain in handling dialectal diversity, context sensitivity, fuzzy sentiments, and code-mixed data. These findings highlight the need for language-specific adaptations and hybrid approaches that integrate advanced deep learning architectures with feature-rich preprocessing pipelines to achieve robust sentiment analysis for non-English languages.

III. Dataset Overview

The dataset considered for this study comprises a training set with 4,352 samples and a testing set with 544 samples, totaling 4,896 labeled instances. This corpus intended for multi-class classification with each comment labeled with a specific category that reflects its sentiment, tone, or intent. These categories include Negative (criticism or disapproval), Positive (approval or agreement), Substantiated (well-supported claims), Opinionated (strong personal views), Sarcastic (irony or mockery), Neutral (unbiased or factual), and None of the above for comments that do not fit any of the defined labels.

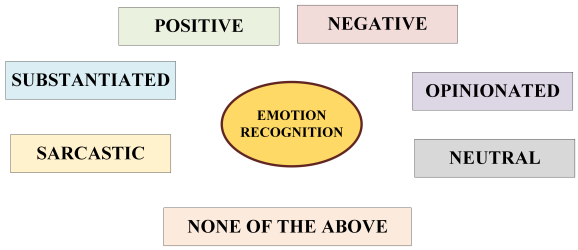

Fig. 1. Different Classes in the Datset

IV. Proposed Methodology

The methodology for multi-class emotion classification of text encompasses following key stages: data cleansing, feature extraction using BERT embeddings, class balancing through the SMOTE-Tomek technique, machine learning model training, and performance evaluation as shown in the Fig. 2.

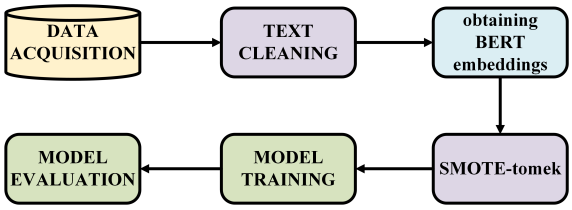

Fig. 2. Proposed Methodoogy for multi-class Classification

A. Dataset cleaning

In the context of political sentiment analysis, cleaning the text dataset is a crucial step that directly impacts the accuracy and reliability of the classification models. Social media, news comments, or public forums tend to be noisy, informal, and diverse in structure. Hence, the text corpus is put through rigorous step by step preprocessing to enhance the clarity and consistency of text data employed for classification as shown in the algorithm I. Missing and non-string values with empty strings are substituted to make it consistent. Irrelevant components such as URLs, email addresses, and user mentions are removed since they typically do not carry meaningful sentiment or stance-related information. Further it involves eliminating unwanted whitespaces, punctuation mark, converting multiple spaces to single spaces, and user mentions through regular expressions. These operations are intended to remove noise and unwanted elements from text. Such thorough preprocessing eases the learning process for machine learning models by providing clean, meaningful, and standardized input, which is essential for achieving consistent and reliable performance in sentiment classification tasks.

B. Feature Extraction using mbert-embeddings

The Multilingual Bidirectional Encoder Representations from Transformers (mBERT) is a transformer-based architecture pre-trained on textual data from over 100 different

Text Preprocessing Steps
1. Check if the input is a string; if not, return an empty string.
2. Strip leading and trailing whitespace from the input text.
3. Replace multiple spaces or newline characters with a single space.
4. Remove URLs and web links from the text.
5. Remove all user mentions (e.g., @username).
6. Return the cleaned and normalized text.

TABLE I
STEPS INVOLVED IN THE TEXT PREPROCESSING FUNCTION

languages. It is designed to capture both contextual and semantic nuances across diverse linguistic structures, making it particularly suitable for multilingual natural language processing tasks. In this study, mBERT embeddings are employed to mathematically represent the textual input in a dense, high-dimensional vector space. Each input sentence is first tokenized and then passed through the mBERT model, from which the [CLS] token embedding is extracted as a fixed-length representation of the sentence. This embedding captures the global semantic essence of the input while retaining syntactic cues. These rich contextual representations are leveraged as features for downstream tasks such as sentiment and emotion classification, where precise understanding of tone, polarity, and intent is critical.

C. Class Imbalance Handling

A critical challenge with the dataset is its uneven class distribution, where certain categories contain significantly more samples than others. For instance, the ”Opinionated” category overwhelmingly dominates the dataset, with over 1,300 instances, as shown in Fig. 3. In contrast, categories such as ”None of the above” and ”Negative” are markedly underrepresented, with fewer than 200 and around 400 samples, respectively. This skewed distribution can severely impact the performance of machine learning models, which tend to overfit to majority classes, resulting in poor classification accuracy on minority categories. This is particularly problematic in nuanced tasks such as sentiment and stance classification, where accurately detecting underrepresented tones like sarcasm or substantiated factual claims is essential for reliable interpretation.

To address this imbalance, an oversampling technique such as the Synthetic Minority Over-sampling Technique (SMOTE) can be applied. Unlike simple duplication, SMOTE generates synthetic samples by interpolating between existing minority class instances, thereby enriching the feature space, improving class representation, and reducing the risk of overfitting. However, solely oversampling may not be sufficient. In cases where categories have subtle semantic differences, such as between ”Opinionated” (strong personal views) and ”Substantiated”, excessive synthetic replication can distort class boundaries or introduce noise.

To mitigate this, SMOTE is combined with Tomek Links in a hybrid strategy known as SMOTE-Tomek. Tomek Links identify and remove ambiguous or overlapping samples from the majority class that lie near decision boundaries, thereby refining the class separation. This combination not only balances the dataset but also enhances the quality of the decision boundaries, ensuring that semantically distinct classes remain well-defined. As a result, SMOTE-Tomek enables more accurate and reliable classification, particularly in complex, imbalanced datasets such as those involving political sentiment analysis.

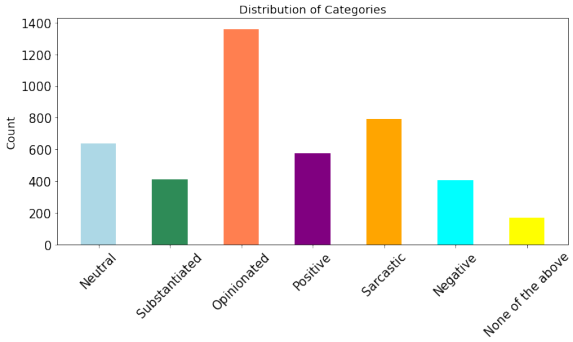

Fig. 3. Class imbalance in the Dataset

D. Model Training

For this study, three distinct classification models have been employed such as Support Vector Machine (SVM), Extreme Gradient Boosting (XGBoost), and Random Forest. These models were chosen for their strong performance on high-dimensional data and their ability to handle complex decision boundaries.

1) Support Vector Machine (SVM): Support Vector Machine (SVM) is a supervised learning algorithm widely used for classification tasks, particularly effective in high-dimensional spaces. The core idea of SVM is to find the optimal hyperplane that maximally separates classes by maximizing the margin between data points of different classes. This margin is defined as the distance between the hyperplane and the closest data points from each class, known as support vectors. By focusing only on these support vectors, SVM achieves a compact and generalizable model. In scenarios where the data is not linearly separable, SVM employs kernel functions to implicitly map the input features into a higher-dimensional space, enabling linear separation in that transformed space. Commonly used kernels include linear, polynomial, and radial basis function (RBF) kernels, among others. This flexibility makes SVM well-suited for handling complex and non-linear classification boundaries. Mathematically, the decision function for an SVM classifier is given in the Eq. (1).

$$f(x) = \text{sign}\left(\sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b\right) \tag{1}$$

where α_i are the learned Lagrange multipliers, y_i are class labels, $K(x_i, x)$ is the kernel function, and b is the bias term.

SVM's regularization capability through the penalty parameter C helps in controlling the trade-off between maximizing the margin and minimizing classification error. Its robustness

to overfitting, especially in sparse and high-dimensional data like text embeddings, makes it a strong candidate for sentiment and emotion classification tasks.

2) Extreme Gradient Boosting Classifier (XGBoost): XGBoost (Extreme Gradient Boosting) is an ensemble learning method based on gradient boosting. It handles numerical and categorical data by constructing sequential decision trees. Each tree is trained using weighted independent variables, where misclassified variables receive higher weights and are passed to the next tree for further refinement. These individual predictors collectively form a strong and accurate model. The model is initialized by assigning weights to all independent variables and defining an objective log loss function. Then, sequential decision trees are constructed to correct the errors of their predecessors. At each step, the model computes the gradient (first derivative) and Hessian (second derivative) of the loss function to determine the direction and magnitude of the updates. The trees are built by splitting features in a way that maximizes the reduction in the objective function. The gain from splitting at a particular node is calculated as shown in the Eq. (2).

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma \quad (2)$$

where G and H represent the sum of gradients and Hessians for the left (L) and right (R) child nodes, λ is the L2 reglarization parameter, and γ is the pruning term that prevents unnecessary splits. If the gain falls below γ, the split is discarded. *Adaptive Re-weighting* Every time a tree is constructed, the weights of misclassified samples are updated, ensuring their higher importance in the next iteration. The predictions are refined iteratively using a learning rate (η) to control the step size, preventing overfitting. The update rule for predictions is given in Eq. (3).

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta T_t(z) \quad (3)$$

where $T_t(z)$ is the output of the current decision tree. The final prediction is obtained by aggregating the weighted contributions from all trees as given in Eq. (4).

$$\hat{y} = \sum_{t=1}^{b} \eta T_t(z) \quad (4)$$

Finally, a softmax function is applied to convert the final scores into class probabilities.

3) Random Forest Classifier: Random Forest is a powerful ensemble learning algorithm for classification tasks, leveraging multiple decision trees to enhance accuracy and mitigate overfitting. It builds an ensemble of decision trees using bootstrap sampling (bagging) and introduces randomness in feature selection, ensuring diversity in tree structures and reducing correlation among them.

Each tree is trained on approximately 63.2% of the original dataset (due to bootstrap sampling), while the remaining 36.8% (out-of-bag data) is used for performance estimation. For each split in a tree, a random subset of $\sqrt{m}$ features (where m is the total number of features) is considered, preventing dominant features from biasing the model. The final classification decision is obtained through majority voting, where each tree casts a vote for a class, and the class with the most votes is assigned as the final prediction. This design significantly reduces variance and enhances generalization, making Random Forest robust against noisy data and feature redundancies.Mathematically, for a classification problem, the final prediction $\hat{y}$ is determined as shown in the Eq. (5).

$$\hat{y} = \arg\max_k \sum_{t=1}^{T} \mathbb{1}(h_t(x) = k) \quad (5)$$

where T is the total number of trees, $h_t(x)$ is the prediction of the t^{th} tree for input x, and k is the class label.

The strength of Random Forest lies in its ability to reduce overfitting compared to a single decision tree while maintaining interpretability. Additionally, it provides feature importance scores by calculating the mean decrease in impurity (Gini index or entropy) across trees, helping identify influential features in classification tasks. Its adaptability across high-dimensional datasets and resistance to noise make it one of the most effective classifiers in machine learning applications.

E. Evaluation Metrics for Classification Tasks

To rigorously assess the performance of the classification models used in this study, several evaluation metrics were employed. These metrics provide insights not only into the overall accuracy but also into how well the model performs across individual classes, especially in the presence of class imbalance.

1) Accuracy: Accuracy is the ratio of correctly predicted samples to the total number of samples. While widely used, it can be misleading in imbalanced datasets, as high accuracy may simply reflect correct classification of the dominant class.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \quad (6)$$

2) Precision:: Precision measures the proportion of correctly predicted positive instances among all instances predicted as positive. It is critical when false positives are costly.

$$\text{Precision} = \frac{TP}{TP+FP} \quad (7)$$

3) Recall (Sensitivity):: Recall measures the proportion of correctly predicted positive instances among all actual positives. It is essential in tasks where false negatives are critical.

$$\text{Recall} = \frac{TP}{TP+FN} \quad (8)$$

4) F1-Score:: The F1-score is the harmonic mean of precision and recall. It provides a balanced measure, particularly useful when the class distribution is skewed.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

5) Macro and Weighted Averages:: For multi-class classification, metrics such as precision, recall, and F1-score are often computed using macro and weighted averaging.

- **Macro Average:** Calculates the metric independently for each class and then takes the average, treating all classes equally regardless of their size.
- **Weighted Average:** Takes the average of the metric across all classes, weighted by the number of instances in each class. This gives a more realistic assessment of performance in imbalanced datasets.

6) Confusion Matrix:: A confusion matrix is a tabular representation showing the actual versus predicted classifications. It helps visualize the types of errors made by the classifier and assess class-wise performance.

These metrics collectively offer a comprehensive evaluation of model effectiveness, especially in multi-class imbalanced classification problems such as sentiment or emotion analysis in Tamil text.

V. Results and Evaluation

A. Classification using Translated Dataset

The Tamil sentiment classification pipeline initially involved translating the original Tamil texts into English using Google Translate. These translated sentences were then processed using a standard English BERT model to obtain sentence embeddings, typically via the [CLS] token or pooled representations. The resulting embeddings were fed into classifiers such as SVM, Random Forest, and XGBoost for emotion classification as shown in Fig. 4. However, the pipeline exhibited severe overfitting, as evidenced by XGBoost achieving a training accuracy of 99.71% while the testing accuracy dropped to 34.79% as shown in Fig. 5. This discrepancy can be attributed to several factors. Primarily, the translation process introduced a significant loss of emotional and cultural nuance inherent in the original Tamil texts. Google Translate often fails to accurately capture tone, sarcasm, and contextual subtleties, particularly in politically charged or idiomatic content. This semantic drift reduced the representational fidelity of the embeddings. Moreover, the English BERT model is not fine-tuned for handling translated content from low-resource languages, further diminishing embedding quality. As a result, the classifiers were trained on feature representations that did not reliably encode the original emotional intent, leading to poor generalization performance on unseen data.

B. Classification with mbert embedding for the Tamil Dataset

The original Tamil dataset was directly utilized without translation by employing multilingual BERT (mBERT), which supports Tamil natively. Each Tamil sentence was tokenized and passed through the mBERT model to extract sentence-level embeddings, typically using the [CLS] token representation from the final hidden layer. These embeddings served as input features for three different traditional classifiers: Support Vector Machine (SVM), Random Forest, and XGBoost. While all classifiers were trained and evaluated, XGBoost achieved the highest training accuracy at 98.82% as shown in Fig. 6 , significantly outperforming the others on the training set.

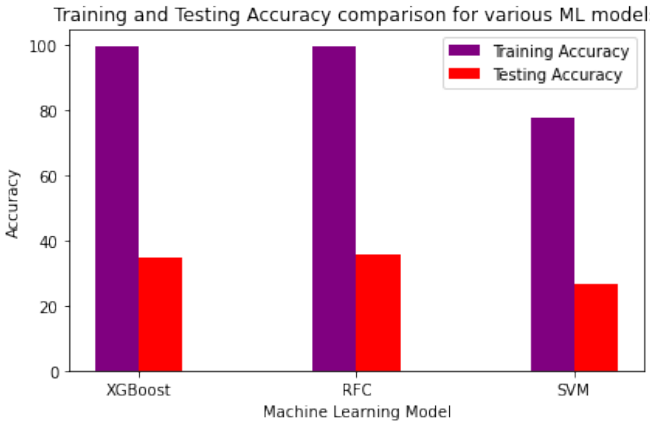

Fig. 4. The Training and Testing accuracy comparison for different ML models.

```
Training Accuracy: 0.9971
Testing Accuracy: 0.3479

Category            precision  recall     f1-score   support
Negative            0.24       0.05       0.08       82
Neutral             0.16       0.10       0.12       117
None of the above   0.72       0.89       0.80       37
Opinionated         0.37       0.67       0.48       282
Positive            0.35       0.18       0.24       120
Sarcastic           0.33       0.27       0.30       159
Substantiated       0.00       0.00       0.00       74

accuracy            0.35       0.35       0.35       2613
macro avg           0.31       0.31       0.29       871
weighted avg        0.30       0.35       0.30       871

Overall Precision: 0.3037
Overall Recall: 0.3479
Overall F1-Score: 0.3011
```

Fig. 5. Classification Performance report for Translated Dataset.

However, the testing accuracy of XGBoost dropped drastically to 29.04%, indicating severe overfitting. This suggests that the model had memorized the training data but failed to generalize to unseen examples. The likely cause is the high model complexity and the lack of regularization during training. To address this overfitting, incorporating L1 (Lasso) or L2 (Ridge) regularization into the XGBoost training configuration is recommended. These techniques penalize overly complex models by shrinking less informative feature weights, which can enhance generalization performance and stabilize predictions on the test set.

C. Classification after Applying Regularisation Techniques

L1 and L2 regularization are applicable only to linear models such as SVMs and logistic regression, as they operate directly on learned weight coefficients. In contrast, ensemble classifiers like Random Forest and XGBoost rely on decision tree structures and do not use weight vectors; their regularization is achieved through tree-specific hyperparameters such as maximum depth, number of estimators, and minimum samples per split.

```
Training Accuracy: 0.9882
Testing Accuracy: 0.2904
                   precision    recall  f1-score   support

         Negative       0.13      0.13      0.13        46
          Neutral       0.16      0.13      0.14        70
None of the above       0.95      0.84      0.89        25
      Opinionated       0.36      0.45      0.40       171
         Positive       0.20      0.19      0.19        75
        Sarcastic       0.27      0.24      0.25       106
    Substantiated       0.13      0.12      0.12        51

         accuracy                           0.29       544
        macro avg       0.32      0.30      0.31       544
     weighted avg       0.28      0.29      0.28       544

Overall Precision: 0.2822
Overall Recall: 0.2904
Overall F1-Score: 0.2841
```

Fig. 6. Classification report upon feeding mbert embeddings into XGBoost classifier.

The mBERT embeddings of the Tamil dataset were first used with a Support Vector Machine (SVM) classifier employing L1 regularization. The objective was to mitigate overfitting by promoting sparsity in the feature weights. However, the model achieved a low training accuracy of 35.63% and a testing accuracy of 25.00% as shown in Fig.7, indicating significant underfitting. The sparsity enforced by L1 regularization likely caused the model to discard relevant features, limiting its ability to learn the underlying patterns in the data. As a result, this approach did not perform well for the task of sentiment classification.

```
Training Accuracy: 0.3563
Testing Accuracy: 0.2500

Classification Report:
                   precision    recall  f1-score   support

         Negative       0.23      0.22      0.22        46
          Neutral       0.18      0.10      0.13        70
None of the above       0.44      0.96      0.60        25
      Opinionated       0.40      0.17      0.24       171
         Positive       0.19      0.28      0.23        75
        Sarcastic       0.27      0.35      0.30       106
    Substantiated       0.09      0.16      0.12        51

         accuracy                           0.25       544
        macro avg       0.26      0.32      0.26       544
     weighted avg       0.27      0.25      0.24       544


L1 Regularization Analysis:
Number of non-zero coefficients: 143
Sparsity ratio: 79.57%

Overall Precision: 0.2750
Overall Recall: 0.2500
Overall F1-Score: 0.2390
```

Fig. 7. Classification report upon feeding mbert embeddings into SVM with L1 Regularization.

Next, the same mBERT embeddings were processed using an SVM classifier with L2 regularization, which instead penalizes large weight magnitudes to improve generalization. Despite this adjustment, the model produced even lower training accuracy at 43.36%, with a 26.29% testing accuracy as shown in Fig. 8. This further confirms that SVM, even with regularization, struggles to model the complexity of high-dimensional BERT-based embeddings.

```
Training Accuracy: 0.4336
Testing Accuracy: 0.2629
Classification Report:
                   precision    recall  f1-score   support

         Negative       0.18      0.30      0.23        46
          Neutral       0.22      0.14      0.17        70
None of the above       0.74      0.92      0.82        25
      Opinionated       0.41      0.17      0.24       171
         Positive       0.23      0.37      0.28        75
        Sarcastic       0.26      0.27      0.27       106
    Substantiated       0.12      0.20      0.15        51

         accuracy                           0.26       544
        macro avg       0.31      0.34      0.31       544
     weighted avg       0.30      0.26      0.26       544

Overall Precision: 0.2996
Overall Recall: 0.2629
Overall F1-Score: 0.2594
```

Fig. 8. Classification report upon feeding mbert embeddings into SVM with L2 Regularization.

VI. Challenges

Sentiment analysis in Tamil presents several inherent challenges rooted in the linguistic characteristics of the language. Tamil is an agglutinative language with complex morphological structure, making tokenization, stemming, and overall preprocessing significantly more difficult compared to languages like English. Generic NLP tools often struggle to handle such complexity without language-specific customization. Additionally, political and opinion-based Tamil text frequently contains sarcasm, irony, and implicit sentiment, which are difficult to detect without deep contextual modeling. Shallow models or basic feature extraction techniques often fail to capture these subtleties, reducing classification accuracy.

In addition to linguistic difficulties, practical issues also impact performance. One major issue is class imbalance in the dataset, where certain sentiment categories are underrepresented. This imbalance can bias models toward the dominant classes, resulting in poor generalization. Techniques such as oversampling, class weighting, and balanced loss functions are often necessary to address this. Furthermore, attempts to translate Tamil data into English to utilize pretrained English models can introduce semantic distortions, which reduce the effectiveness of embedding-based models. Computational constraints also play a role, as BERT-based models like mBERT require significant processing power for both training

and inference, posing challenges for large-scale or real-time sentiment analysis in resource-limited settings.

VII. Conclusion

In conclusion, developing an effective sentiment classification pipeline for Tamil text involves addressing a multifaceted set of challenges spanning linguistic complexity, data limitations, and computational constraints. The agglutinative nature and rich morphology of Tamil require language-specific preprocessing, while the presence of implicit sentiment, sarcasm, and political context calls for advanced models with deep contextual understanding. Data-related issues, such as class imbalance, necessitate careful handling through resampling strategies or balanced loss functions to ensure fair and accurate predictions. Additionally, translation-based approaches introduce semantic drift that can compromise embedding quality, and high computational demands of transformer-based models limit their scalability. These challenges highlight that conventional methods, even those successful in high-resource languages, do not directly transfer to low-resource, morphologically rich languages like Tamil. Therefore, future work must prioritize direct modeling on Tamil script, integration of domain-aware embeddings, and lightweight architectures suited for deployment in resource-constrained environments. Combining linguistic expertise with tailored machine learning strategies is essential to build accurate, generalizable, and scalable sentiment analysis systems for Tamil and other underrepresented languages.

References

[1] F. Djatmiko, R. Ferdiana, and M. Faris, "A review of sentiment analysis for non-english language," in *2019 International Conference of Artificial Intelligence and Information Technology (ICAIIT)*. IEEE, 2019, pp. 448–451.

[2] V. Barriere and A. Balahur, "Improving sentiment analysis over non-english tweets using multilingual transformers and automatic translation for data-augmentation," *arXiv preprint arXiv:2010.03486*, 2020.

[3] A. Elouardighi, M. Maghfour, H. Hammia, and F.-z. Aazi, "A machine learning approach for sentiment analysis in the standard or dialectal arabic facebook comments," in *2017 3rd International Conference of Cloud Computing Technologies and Applications (CloudTech)*. IEEE, 2017, pp. 1–8.

[4] K. V. Reddy, S. Kumar, and K. Soman, "Sentiment analysis at document level of telugu data from multi-domains," in *2023 3rd International Conference on Intelligent Technologies (CONIT)*. IEEE, 2023, pp. 1–8.

[5] K. Nithya, S. Sathyapriya, M. Sulochana, S. Thaarini, and C. Dhivyaa, "Deep learning based analysis on code-mixed tamil text for sentiment classification with pre-trained ulmfit," in *2022 6th International Conference on Computing Methodologies and Communication (ICCMC)*. IEEE, 2022, pp. 1112–1116.

[6] A. Chiorrini, C. Diamantini, A. Mircoli, D. Potena *et al.*, "Emotion and sentiment analysis of tweets using bert." in *Edbt/icdt workshops*, vol. 3, 2021, pp. 1–7.

[7] H. T. Phan, V. C. Tran, N. T. Nguyen, and D. Hwang, "Improving the performance of sentiment analysis of tweets containing fuzzy sentiment using the feature ensemble model," *Ieee Access*, vol. 8, pp. 14 630–14 641, 2020.

[8] P. Durga and D. Godavarthi, "Deep-sentiment: an effective deep sentiment analysis using a decision-based recurrent neural network (d-rnn)," *IEEE Access*, vol. 11, pp. 108 433–108 447, 2023.